# HACK SECURE INTERNSHIP PROGRAM 2026

## BLUE TEAM — TASK 3

**Security Reporting & Risk Documentation**

**Prepared by: Kalla Kushwanth**

**Assessment Environment: Local DVWA Security Testing Lab**

**Target URL: http://localhost:4280**

**Assessment Type: Controlled Security Testing**

**Scope: Brute-force and directory enumeration, plaintext credential exposure, SQL injection (SQLi), and reflected cross-site scripting (XSS).**

Report Status: Draft — Python log-detection evidence included

# 1. Executive Summary

This report documents the security testing performed as part of the Hack Secure Internship Program 2026, Blue Team Task 3: Security Reporting and Risk Documentation.

The assessment was conducted in a local Damn Vulnerable Web Application (DVWA) environment. The testing focused on four security areas: brute-force activity and directory enumeration, plaintext credential exposure, SQL injection (SQLi), and reflected cross-site scripting (XSS).

Wireshark was used to inspect HTTP traffic, and Docker-accessible web server logs were analyzed to identify suspicious request patterns. A lightweight Python log-analysis script was also executed to generate alerts from the saved logs.

The assessment results demonstrate the importance of secure authentication, encrypted communication, safe database query handling, input/output protection, and effective security monitoring. All tests were performed in a controlled local environment.

# 2. Scope and Methodology

## 2.1 Assessment Environment

- **Application:** Damn Vulnerable Web Application (DVWA)
- **Environment:** Local Docker-based testing lab
- **Target URL:** http://localhost:4280
- **Traffic analysis tool:** Wireshark
- **Log source:** Docker-accessible web server logs
- **Detection method:** Python-based log analysis

## 2.2 Scope of Testing

The assessment covered the following activities:

1. Brute-force login attempts and directory enumeration.
2. Identification of credentials exposed through unencrypted HTTP traffic.
3. SQL injection testing using the DVWA SQL injection module.
4. Reflected cross-site scripting testing using the DVWA XSS module.
5. Review of web server logs and generation of detection alerts using a Python script.

## 2.3 Methodology

Testing was conducted against the local DVWA application using controlled test inputs. Wireshark was used to inspect network traffic, while web server logs were reviewed for suspicious requests and relevant URL parameters.

A Python script analyzed the saved log file, `dvwa_task3_logs.txt`, which contained 118 lines. The script searched for patterns associated with repeated login attempts, directory probing, SQL injection tests, and reflected XSS tests.

The assessment was restricted to the local lab. The results represent observed test activity and do not establish that an external system was compromised.

## 2.4 Limitations

The results depend on the available traffic captures, web server logs, application configuration, and detection patterns. The Python script identifies matching patterns and may generate false positives. Its alerts require analyst review.

A dedicated Wazuh deployment was not used. The detection results in this report are from the lightweight Python log-analysis approach.

# 3. Findings and Risk Analysis

## 3.1 Finding 1: Brute-Force Activity and Directory Enumeration

### Description

Brute-force activity involves repeated attempts to guess valid login credentials. Directory enumeration involves requesting common paths to discover available or restricted resources on a web server.

During the local DVWA assessment, repeated requests to the brute-force module were observed in the web server logs. Requests to paths such as `/admin/`, `/config/`, and `/robots.txt` were also recorded.

### Evidence

The collected evidence includes DVWA login-attempt screenshots, web server log entries, and Python detection output. The detection script identified 17 matching login-request entries and 9 matching directory/path-probing entries.

### Potential Impact

If similar behavior occurs against a production application, repeated login attempts may lead to unauthorized account access when authentication controls are weak. Directory probing may reveal application paths or help identify exposed resources.

**Risk Rating:** Medium to High, depending on the authentication controls and sensitivity of the exposed resources.

**Recommended Remediation**

- Apply rate limiting to login endpoints.
- Use account lockout or progressive delays after repeated failed attempts.
- Monitor repeated login failures and unusual request patterns.
- Restrict access to administrative and configuration paths.
- Disable directory listing where it is unnecessary.

## 3.2 Finding 2: Plaintext Credential Exposure

**Description**

During testing, web server log entries showed username and password values in the query string of HTTP requests to the DVWA brute-force module. Because the local application was accessed through HTTP rather than HTTPS, the request data was not protected by TLS encryption.

**Evidence**

The collected web server logs contain request URLs with `username` and `password` parameters. Wireshark captures can provide additional evidence of the HTTP request contents. Credential values should be redacted from report screenshots.

**Potential Impact**

Credentials transmitted through unencrypted HTTP may be visible to parties able to observe the relevant network traffic. Credentials in URLs may also be retained in browser history, access logs, and other records.

The local lab evidence demonstrates exposure in the test environment; it does not prove that any external party intercepted the credentials.

**Risk Rating:** High where real credentials or sensitive information are transmitted.

**Recommended Remediation**

- Enforce HTTPS using TLS.
- Avoid placing credentials in URL query strings.
- Submit credentials through secure request bodies.
- Redact sensitive values from web server logs.
- Use secure password storage and authentication practices.

## 3.3 Finding 3: SQL Injection (SQLi)

**Description**

SQL injection is a vulnerability that occurs when untrusted input is incorporated into a database query without suitable protection. An attacker may manipulate a query to retrieve or alter data beyond the intended operation.

In the local DVWA SQL injection module, a controlled test payload was entered into the input parameter. The application returned multiple records, and the request was visible in the Wireshark capture.

**Evidence**

The evidence includes the DVWA SQL injection test screenshot, the corresponding HTTP request in Wireshark, and relevant web server log entries. The Python detection script identified 12 log entries matching its SQL injection patterns.

**Potential Impact**

In a vulnerable production application, SQL injection may allow unauthorized data access, authentication bypass, or modification of database information, depending on the query and database permissions.

**Risk Rating:** High, subject to validation of the vulnerability and the sensitivity of accessible data.

**Recommended Remediation**

- Use parameterized queries or prepared statements.
- Avoid constructing SQL queries by concatenating user input.
- Validate input according to the expected data type and format.
- Apply least-privilege permissions to database accounts.
- Perform security testing after remediation.

## 3.4 Finding 4: Reflected Cross-Site Scripting (XSS)

**Description**

Reflected XSS occurs when an application includes untrusted input in a response without appropriate output encoding. This can cause browser-side script execution in the context of the application.

During testing, a controlled script-based payload was submitted to the DVWA reflected XSS module. A browser alert appeared, and the corresponding HTTP request was observed in Wireshark.

**Evidence**

The collected evidence includes the browser alert screenshot, the relevant Wireshark request, and web server log entries. The Python detection script identified 8 log entries matching its reflected XSS patterns.

**Potential Impact**

If a similar vulnerability exists in a production application, an attacker may be able to execute script in another user's browser context. Depending on the circumstances, this could enable page manipulation or exposure of information accessible to the affected browser session.

**Risk Rating:** Medium to High, depending on the affected page, user interaction, and available browser protections.

**Recommended Remediation**

- Apply context-appropriate output encoding.
- Validate input where appropriate.
- Avoid inserting untrusted input into executable HTML or JavaScript contexts.
- Use Content Security Policy as an additional defense.
- Retest the application after applying fixes.

# 4. Indicators of Compromise (IoCs)

Indicators of Compromise are observable details that may help identify suspicious activity. The indicators below were observed during controlled testing in the local DVWA environment. They should not be treated as proof of compromise of an external system.

| Finding | Observed indicator | Evidence source |
|---|---|---|
| Brute-force activity | Repeated requests to `/vulnerabilities/brute/` with login parameters | Web server logs |
| Directory enumeration | Requests to `/admin/`, `/config/`, and `/robots.txt` | Web server logs |

| | | |
|---|---|---|
| Plaintext credential exposure | Username and password values in HTTP query strings | Web server logs and Wireshark |
| SQL injection | SQL-like test payload in the `id` parameter of `/vulnerabilities/sqli/` | Wireshark and web server logs |
| Reflected XSS | Script-like test payload in the `name` parameter of `/vulnerabilities/xss_r/` | Browser screenshot, Wireshark, and web server logs |

The source address `172.18.0.1` appears in the logs and belongs to the local Docker networking environment in this setup. It should not be classified as a malicious external IP address.

Credentials and sensitive request parameters must be redacted before screenshots are included in the final report.

# 5. MITRE ATT&CK Mapping

MITRE ATT&CK provides a knowledge base of adversary tactics and techniques. The mappings below describe similarities between the observed test behavior and documented techniques; they do not establish that a real-world intrusion occurred.

| Observed behavior | Technique | ID | Mapping rationale |
|---|---|---|---|

| | | | |
|---|---|---|---|
| Repeated password-guessing requests | Brute Force: Password Guessing | T1110.001 | Different credential combinations were observed in login requests. |
| Requests to common paths | Active Scanning: Wordlist Scanning | T1595.003 | A possible mapping for systematic path probing; the observed requests alone do not prove automated wordlist scanning. |
| SQL injection testing against a vulnerable application | Exploit Public-Facing Application | T1190 | Related exploitation technique, but the tested application was local, not public-facing. This mapping is illustrative only. |
| Reflected XSS test | No direct mapping assigned | — | The evidence demonstrates a web application test; no broader ATT&CK technique is assigned without sufficient supporting behavior. |
| Credentials in HTTP query strings | No direct mapping assigned | — | This is documented as a credential-handling and transport-security weakness. Interception or credential theft was not demonstrated. |

# 6. Security Recommendations

1. **Strengthen authentication:** Implement rate limiting, progressive delays, account lockout controls where appropriate, and monitoring for repeated failed logins.

2. **Restrict unnecessary paths:** Limit access to administrative and configuration resources and disable unnecessary directory listing.
3. **Protect credentials:** Enforce HTTPS, avoid credentials in URLs, and redact sensitive values from logs.
4. **Prevent SQL injection:** Use parameterized queries, validate input, and avoid unsafe query construction.
5. **Prevent XSS:** Apply context-appropriate output encoding and use Content Security Policy as an additional defense.
6. **Improve monitoring:** Retain relevant web server logs and tune detection rules to reduce false positives.
7. **Protect evidence:** Redact credentials and sensitive request parameters before sharing screenshots or reports.
8. **Retest after remediation:** Repeat controlled tests to confirm that security controls work as intended.

# 7. Detection Results and Evidence

## 7.1 Detection Approach

A lightweight Python log-analysis script was executed against `dvwa_task3_logs.txt`. The script analyzed 118 log lines and generated alert summaries for four categories of suspicious activity.

## 7.2 Detection Summary

| Detection category | Matching entries | Script-assigned severity |
|---|---|---|
| Repeated login attempts | 17 | High |

| | | |
|---|---|---|
| Directory/path probing | 9 | Medium |
| Possible SQL injection test | 12 | High |
| Possible reflected XSS test | 8 | High |

These are matching log-entry counts, not necessarily unique attacks. Some entries may overlap between categories. The severity labels are configured by the script and are not formal risk ratings.

## 7.3 Evidence Collected

The evidence set includes:

1. DVWA screenshots showing the controlled login, SQL injection, and reflected XSS tests.
2. Web server logs showing relevant requests and path probing.
3. Wireshark captures showing the corresponding HTTP requests.
4. A Python detection-results screenshot showing the four alert summaries.
5. The output file `task3_detection_alerts.txt`.

The final report should include the actual screenshots beneath the corresponding finding or evidence subsection. Sensitive credential values must be redacted.

## 7.4 Detection Limitations

The Python script performs pattern-based analysis of saved logs. Its alerts indicate matching patterns and require analyst review. HTTP status code 200 alone does not establish that an attack succeeded.

The Python results are log-analysis alerts, not Wazuh alerts. The evidence was collected in a local test environment, and no external system compromise was demonstrated.

# 8. Conclusion

This assessment examined brute-force activity and directory enumeration, plaintext credential exposure, SQL injection, and reflected cross-site scripting in a controlled local DVWA environment.

Wireshark, web server logs, application screenshots, and a Python log-analysis script were used to document the observed activity. The script generated alerts for repeated login requests, path probing, SQL-like input patterns, and script-like input patterns.

The findings highlight the importance of secure authentication, encrypted communication, safe database query handling, appropriate output encoding, and effective monitoring.

The assessment was restricted to the local lab. The results should be interpreted as evidence of controlled testing rather than proof of compromise of an external system. Further validation after remediation is recommended.

# EVIDENCE APPENDIX

Hack Secure Internship Program 2026 — Blue Team Task 3

Screenshots are ordered by finding and evidence type. Each figure has its own page to keep the image readable. The screenshots document controlled activity in the local DVWA lab and do not demonstrate compromise of an external system. Redact credentials before sharing the report.

# Finding 1 — Brute-force / Directory Enumeration

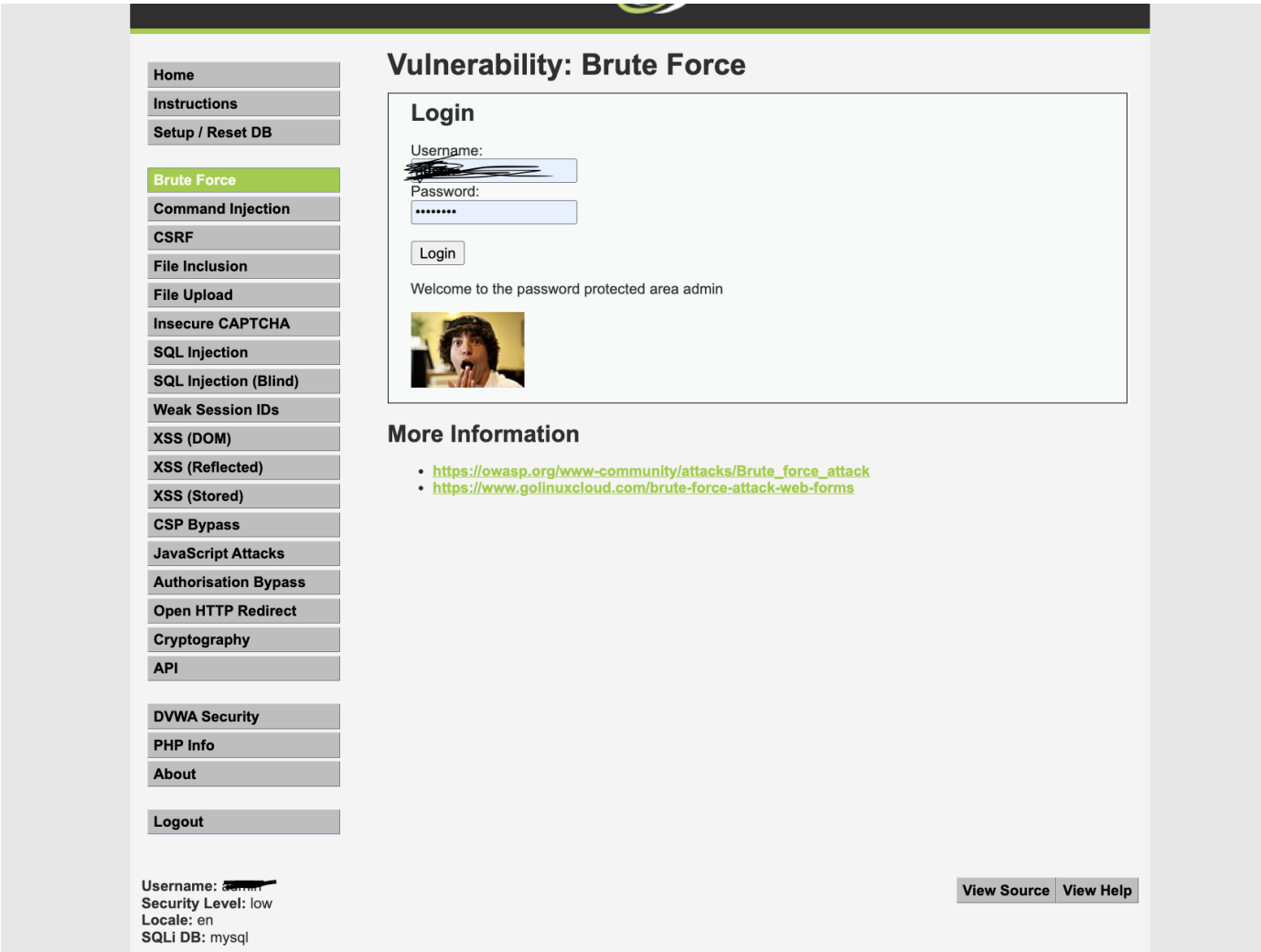



Figure E1. DVWA brute-force module. Shows the login test page used in the controlled local assessment.

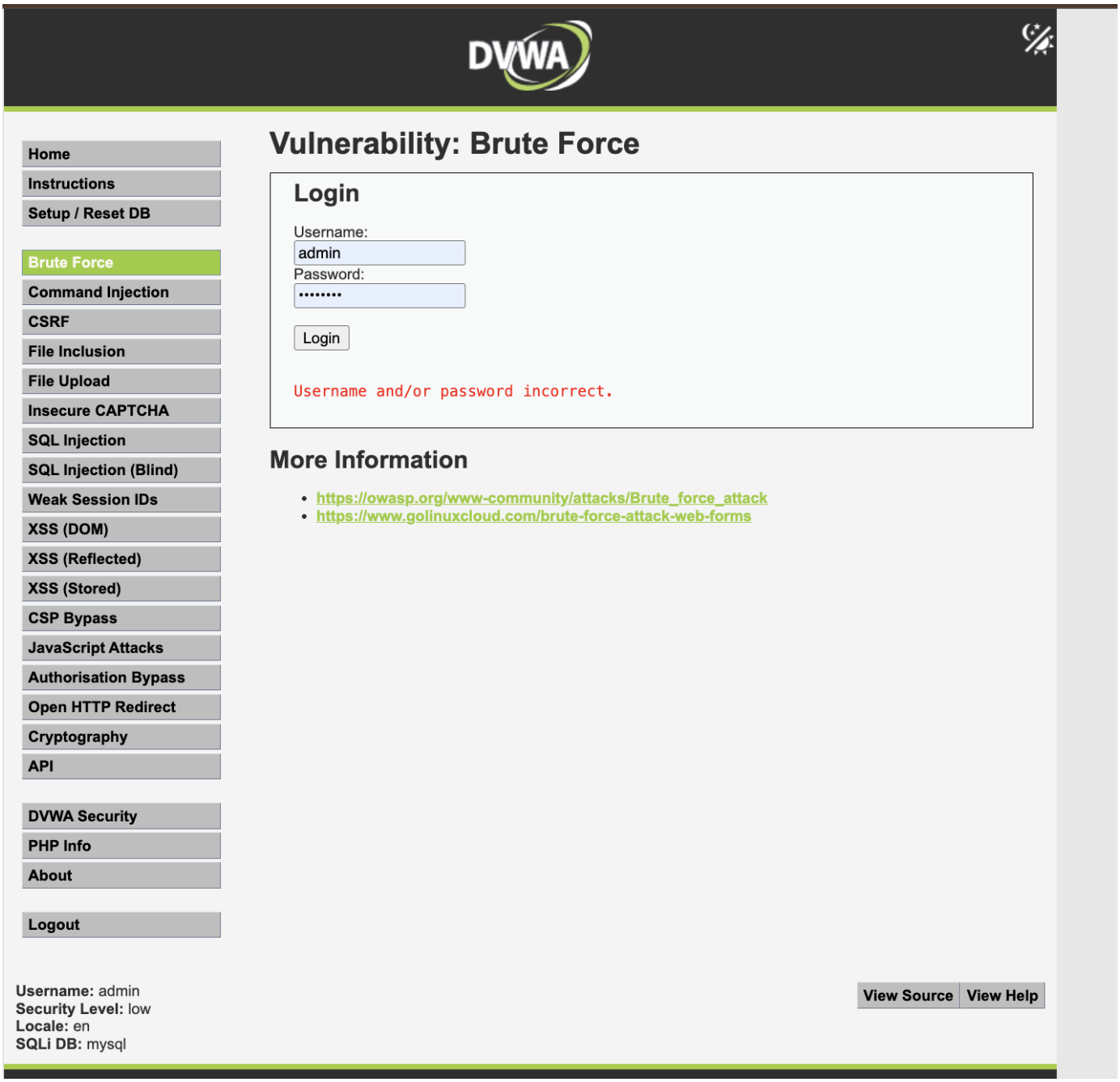



Figure E2. DVWA brute-force result page. Supports the controlled login-attempt test.

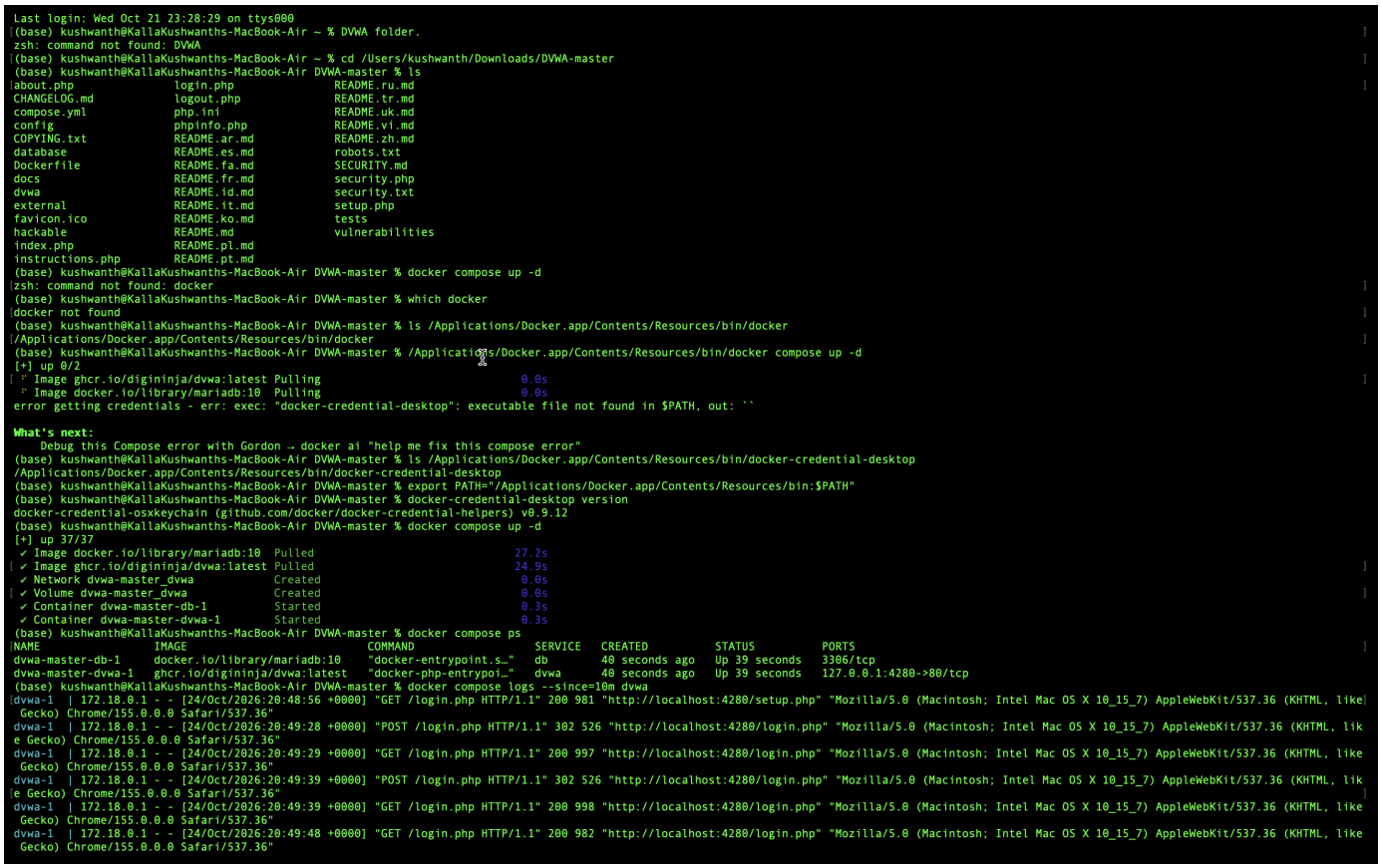



Figure E3. Web server log evidence showing requests to the DVWA brute-force endpoint. Review and redact credential values before external sharing.

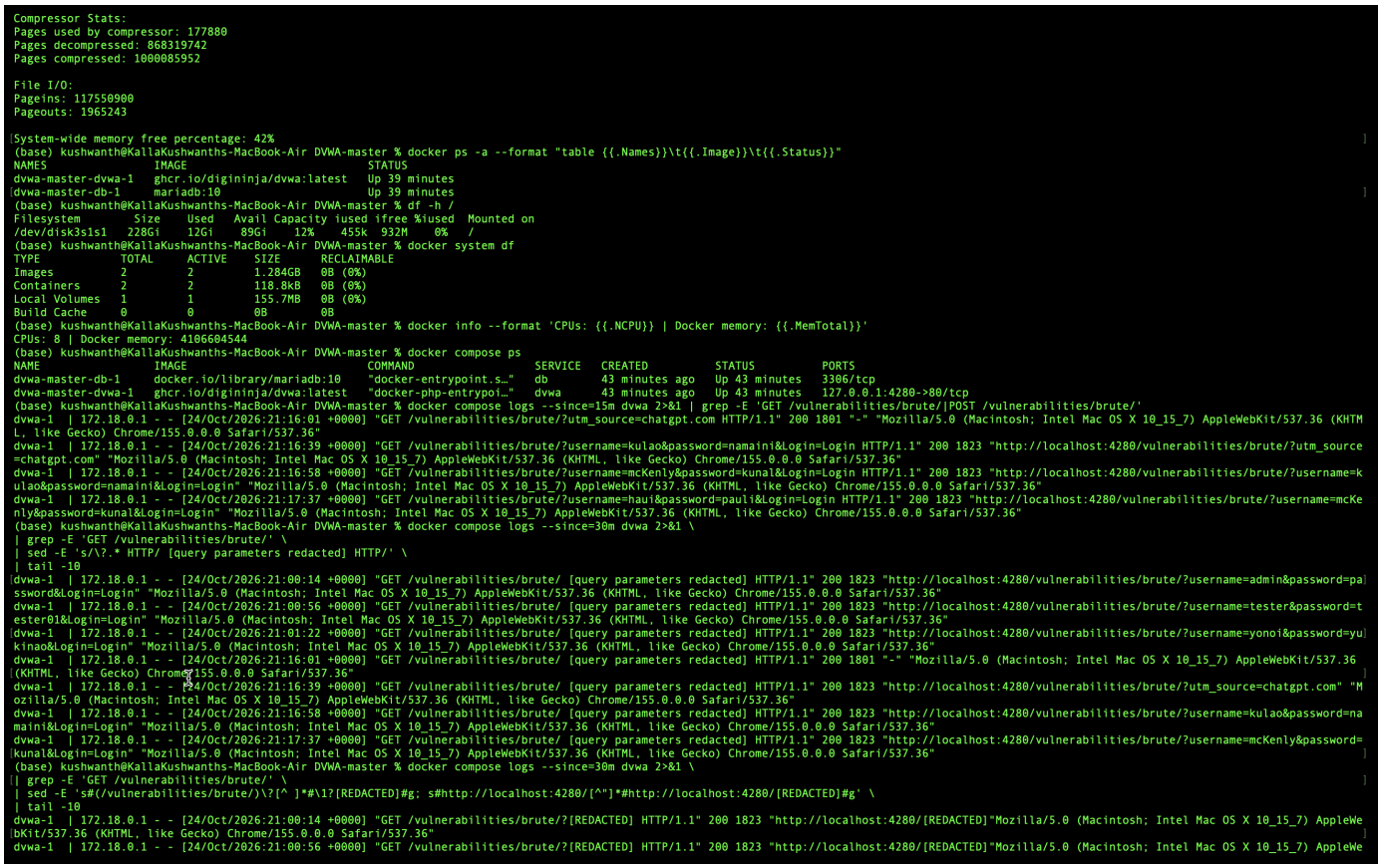



Figure E4. Web server log review showing path and endpoint requests from the local test environment.

# Finding 2 — Plaintext Credential Exposure

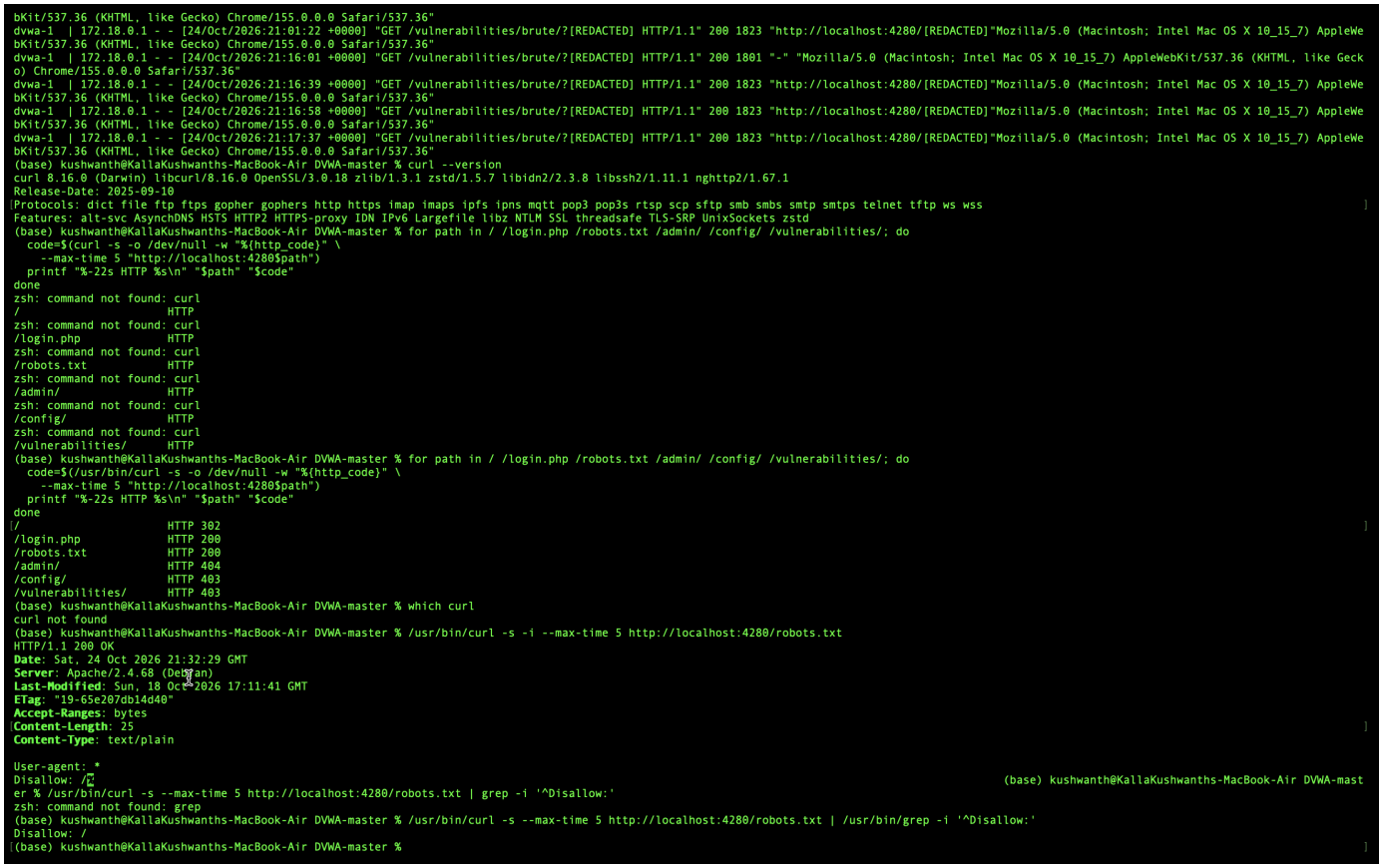



Figure E5. Selected web server log output. Sensitive test values must be redacted in the submitted version.

# Finding 3 — SQL Injection

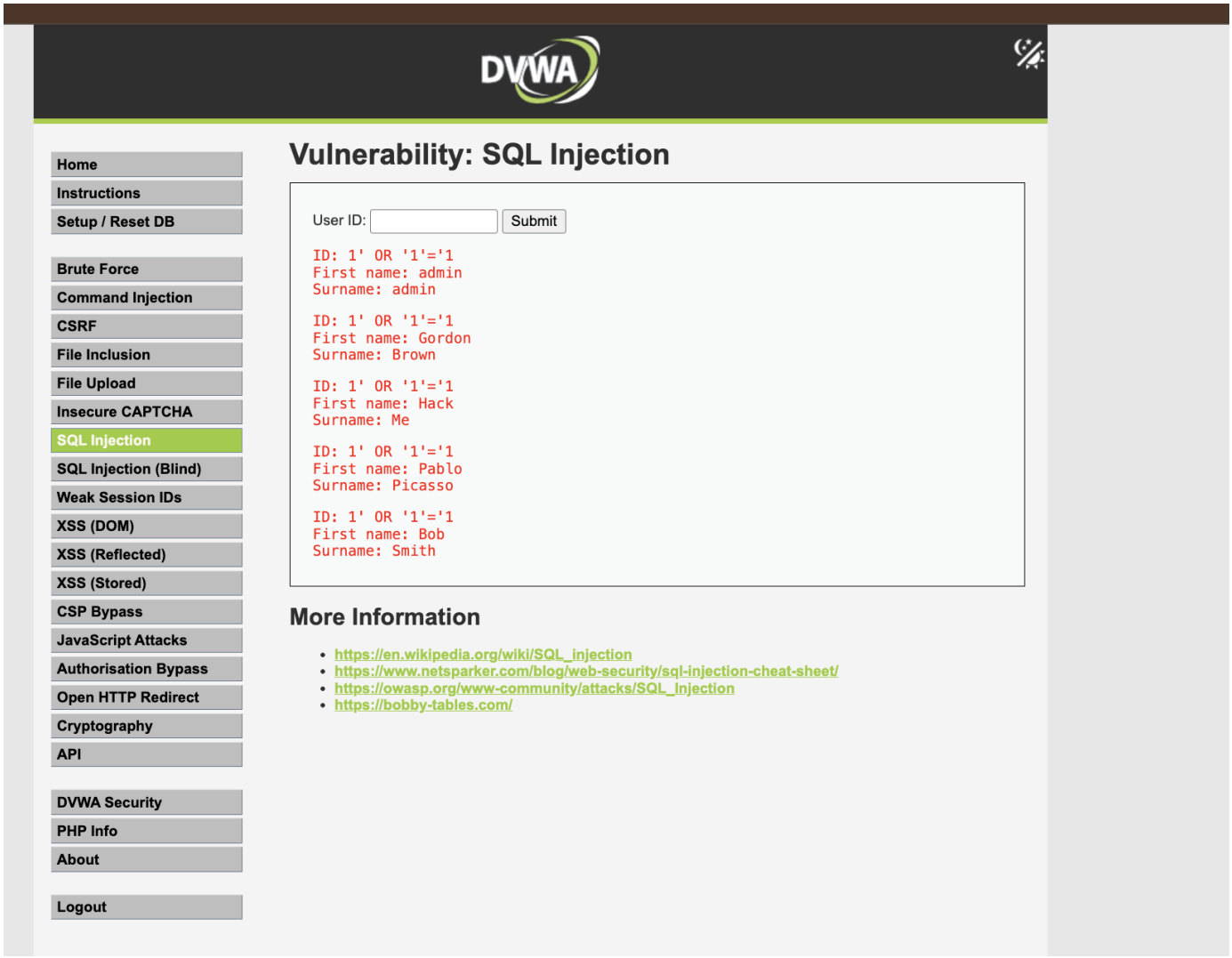



Figure E6. DVWA SQL injection module showing the controlled test result.

## Finding 4 — Reflected XSS

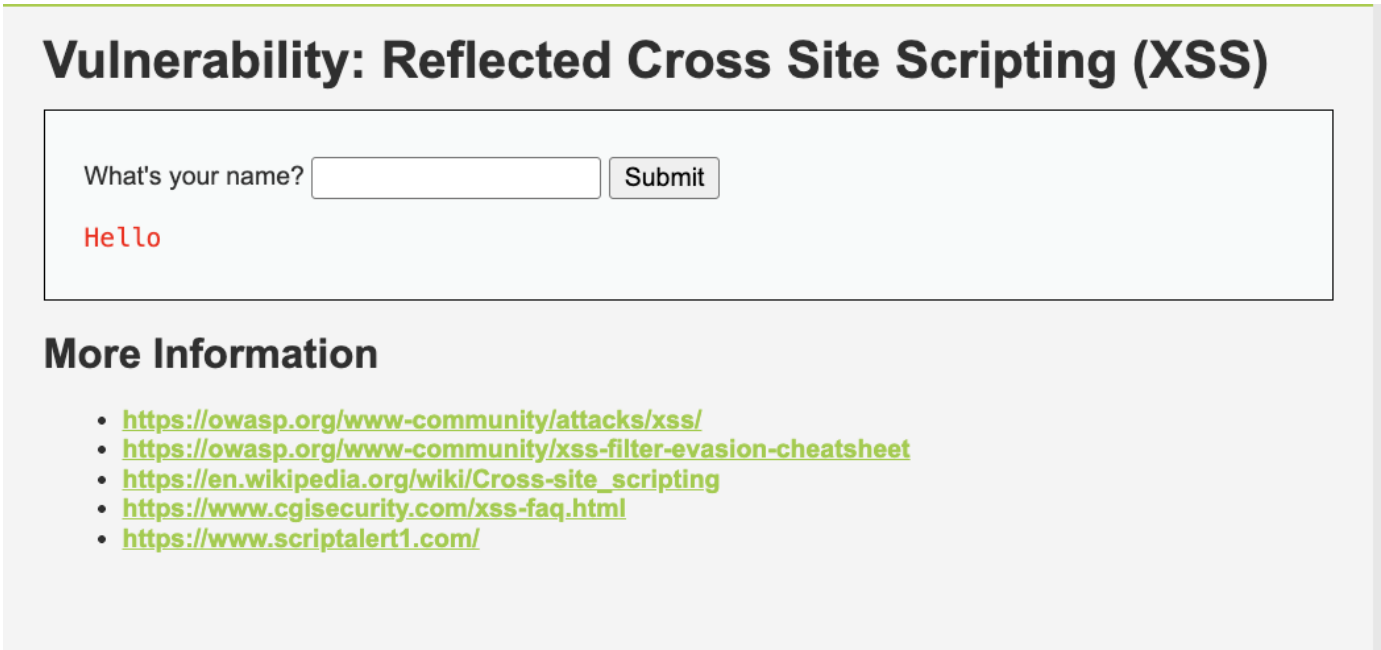



Figure E7. DVWA reflected XSS test page with the submitted test input.

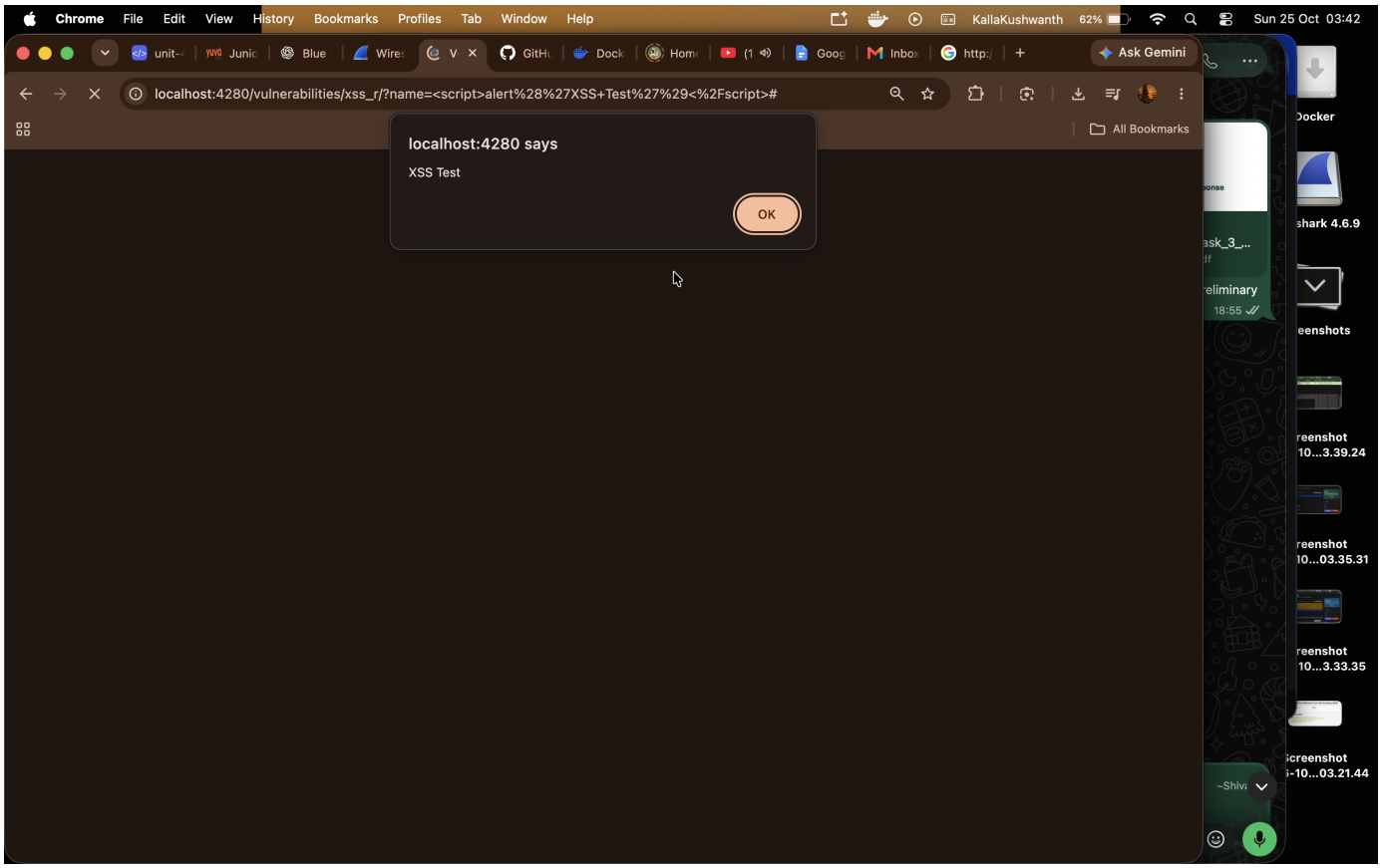



Figure E8. Browser alert shown during the controlled reflected XSS test.

# Network Traffic Evidence — Wireshark

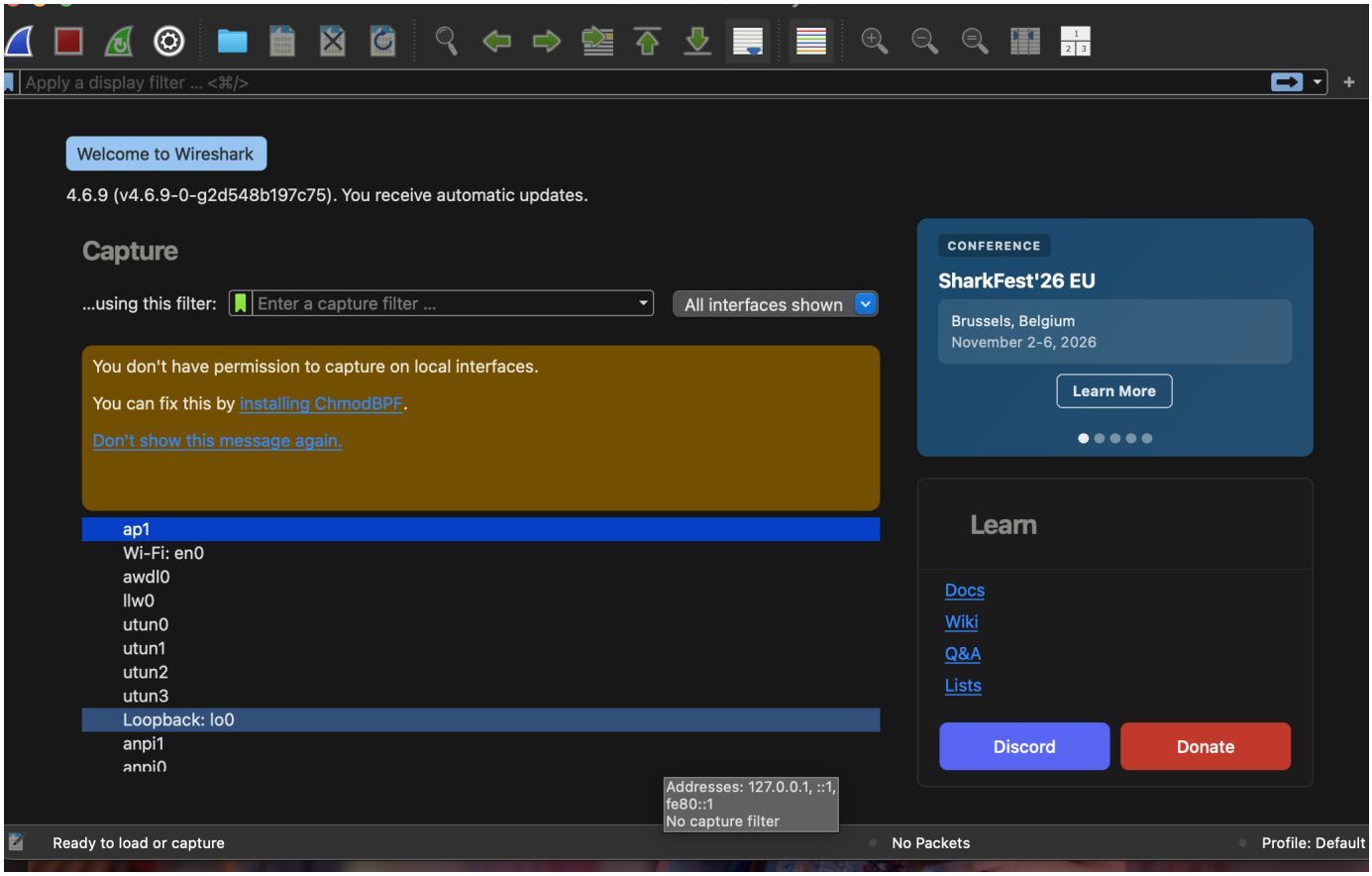



Figure E9. Wireshark capture configuration/interface view. This is supporting setup evidence, not by itself proof of an alert.

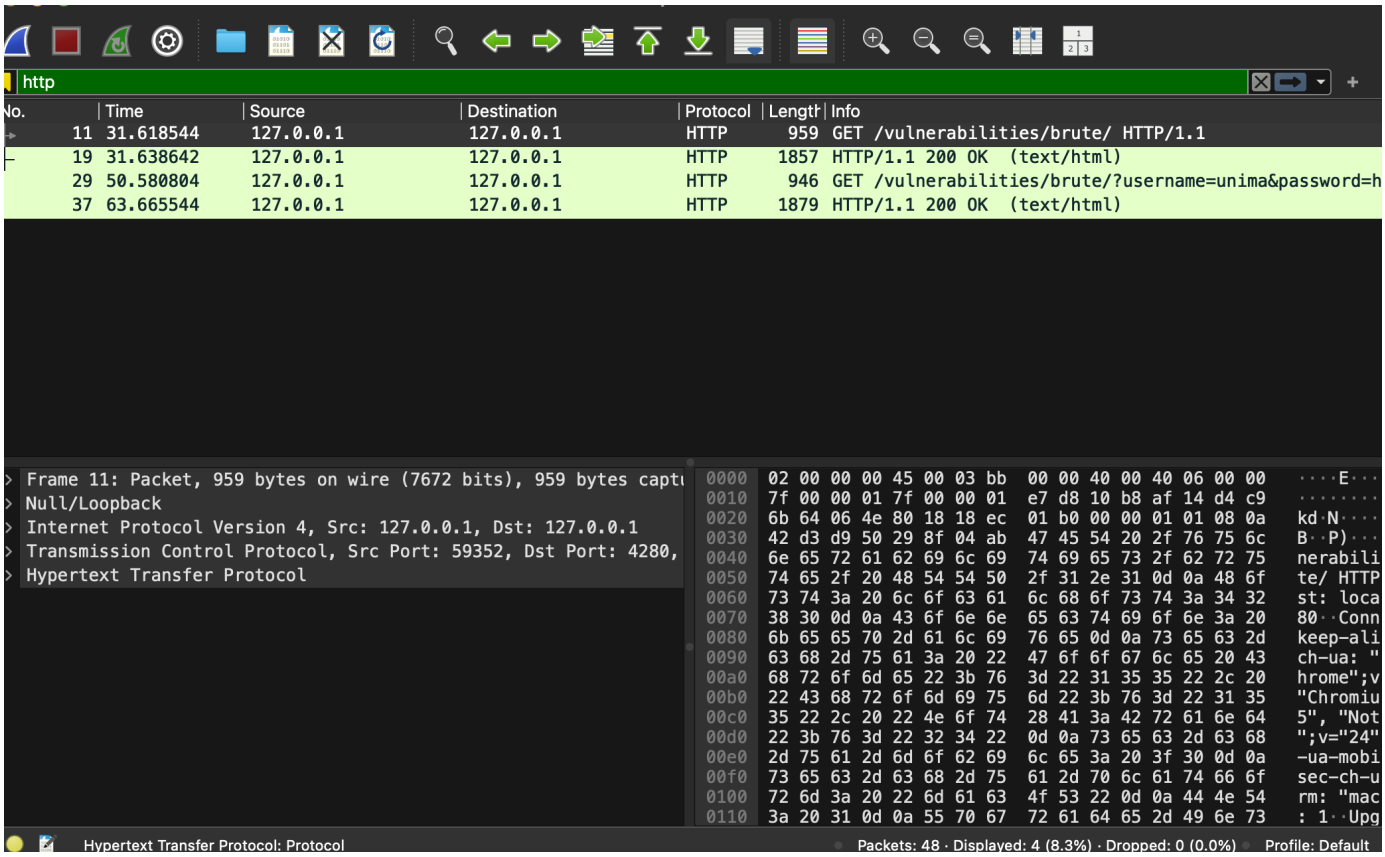



Figure E10. Wireshark packet capture and packet details from the local test environment.

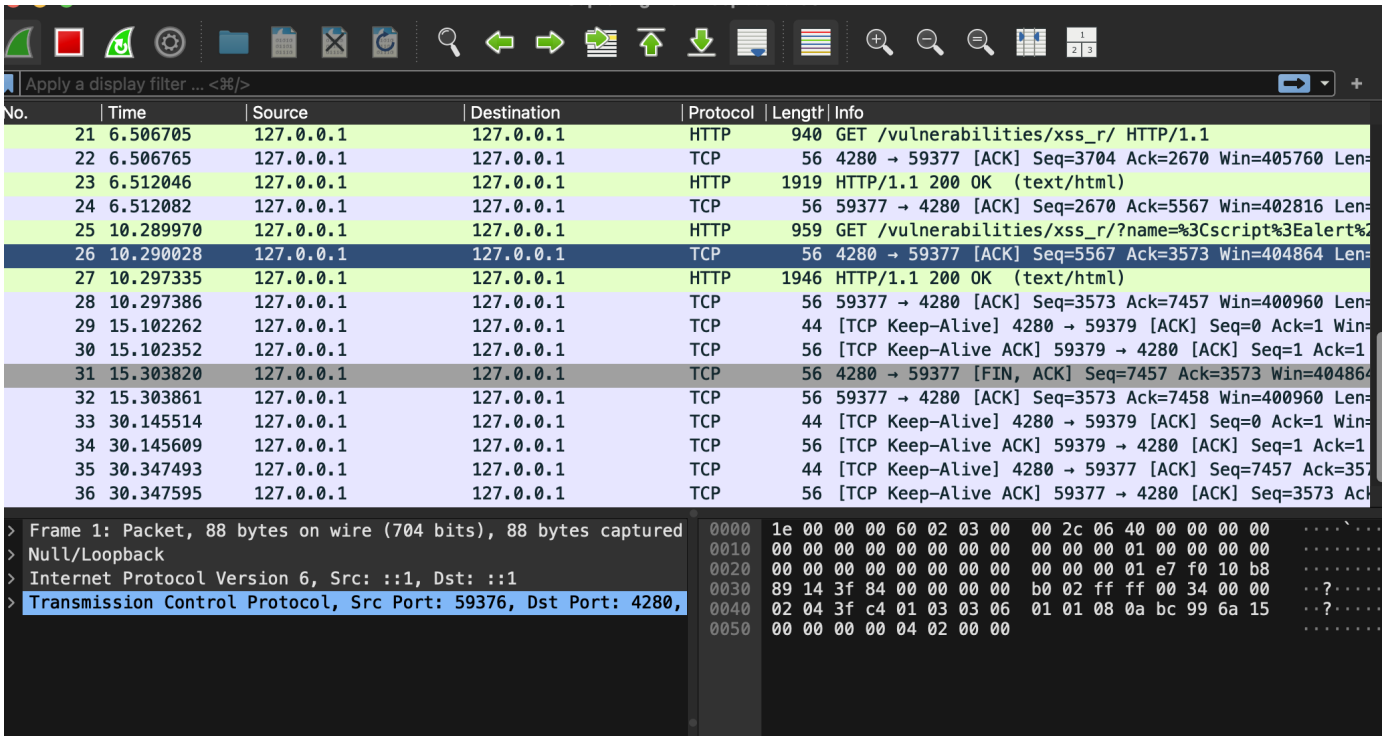



Figure E11. Wireshark packet list and packet details showing captured HTTP traffic.

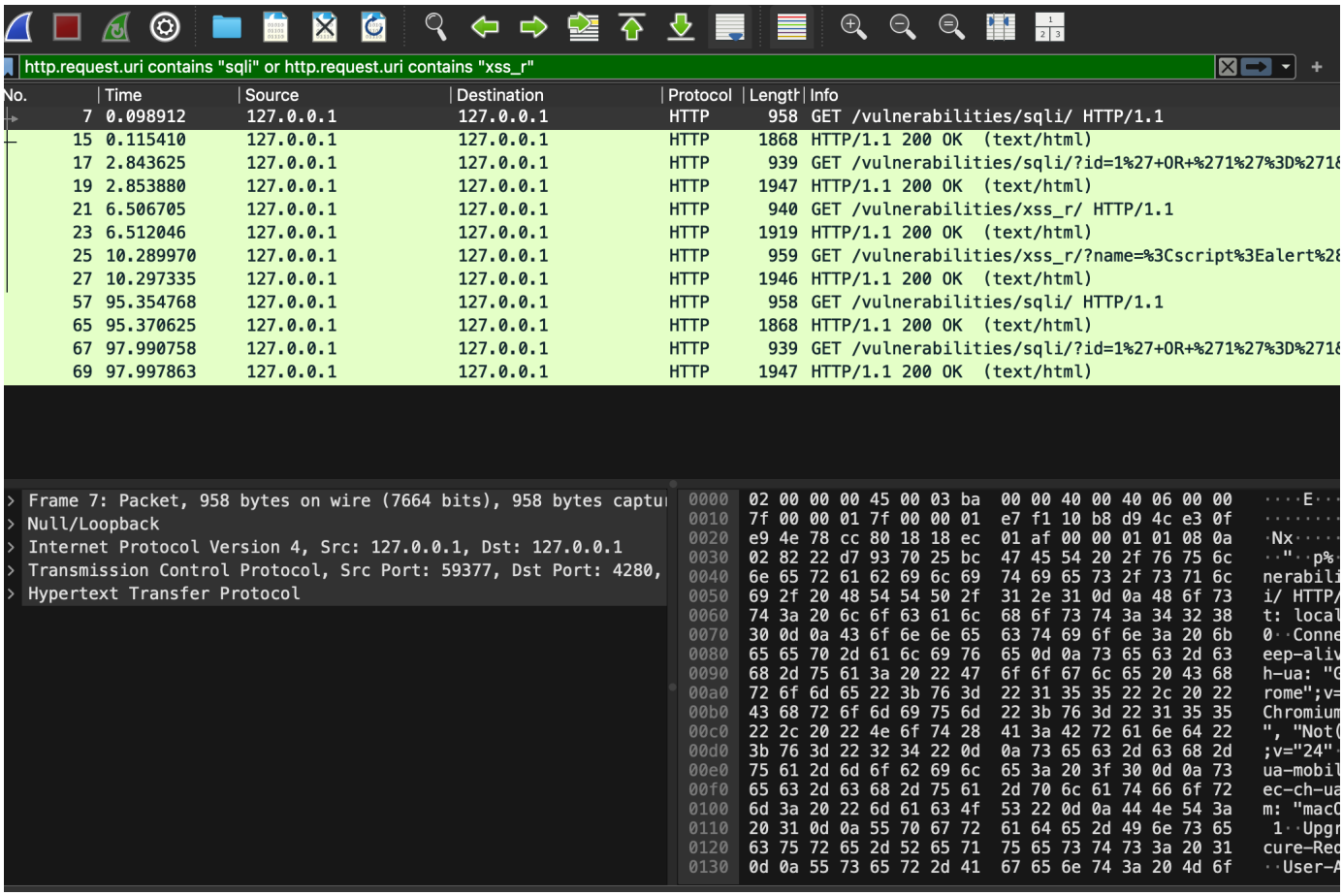



Figure E12. Wireshark packet details for the local HTTP test traffic.

# Python Log Detection

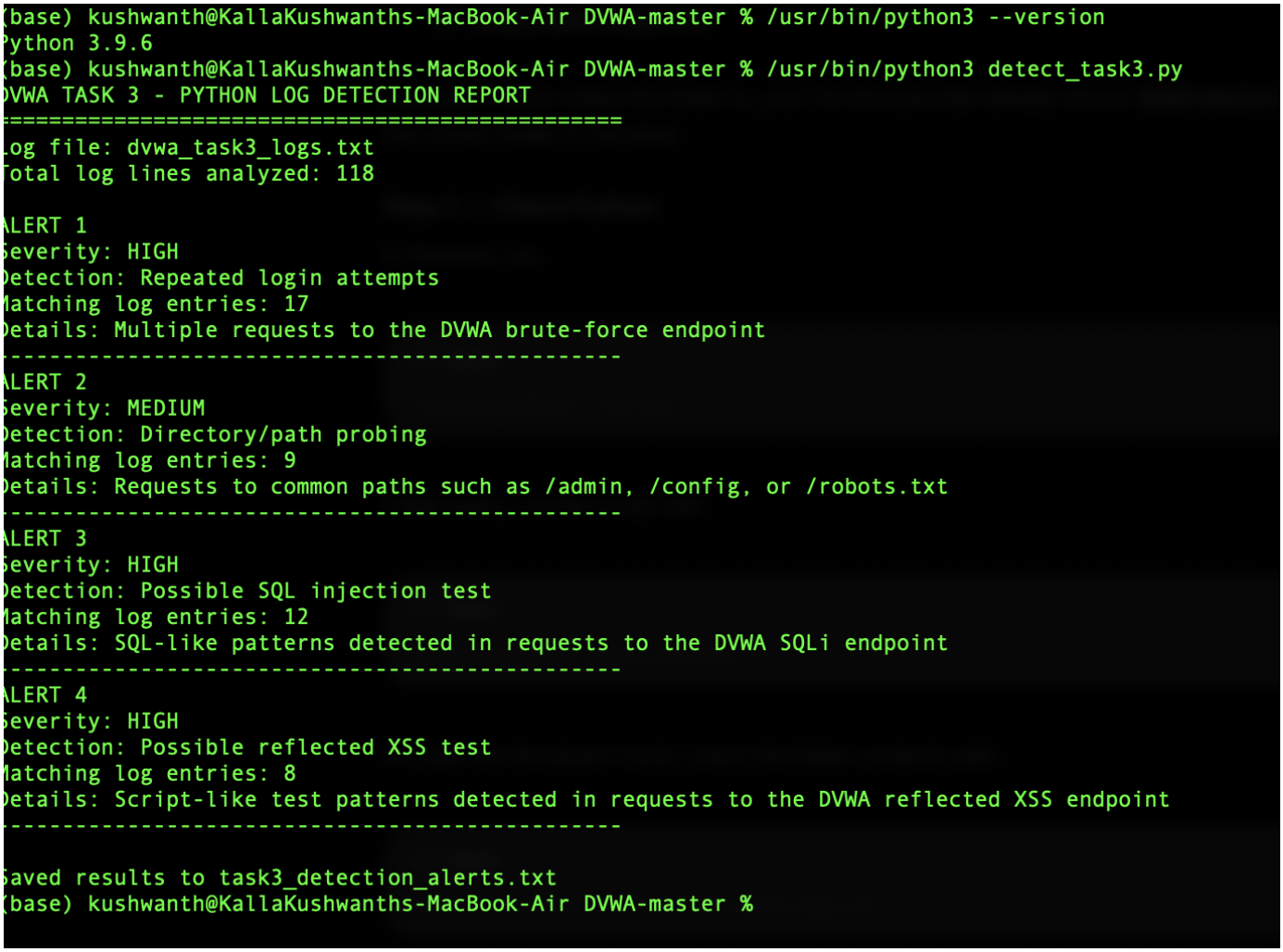



Figure E13. Python log-analysis script output showing four categories of pattern-based alerts from 118 log lines.

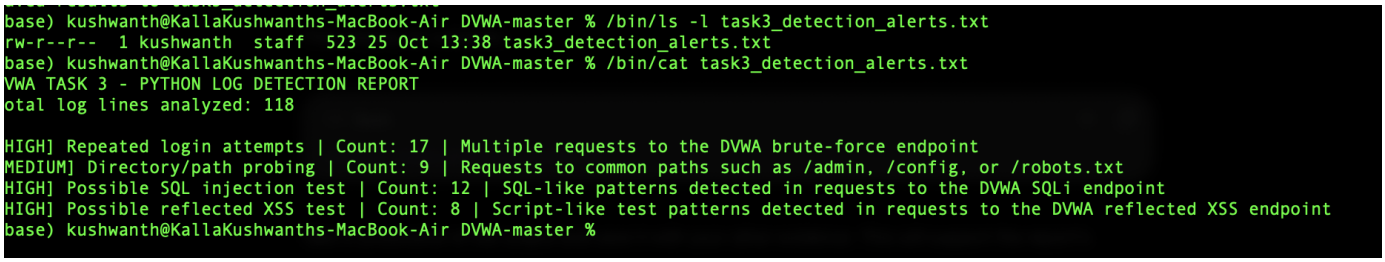



Figure E14. Terminal output associated with the Python log-detection workflow.

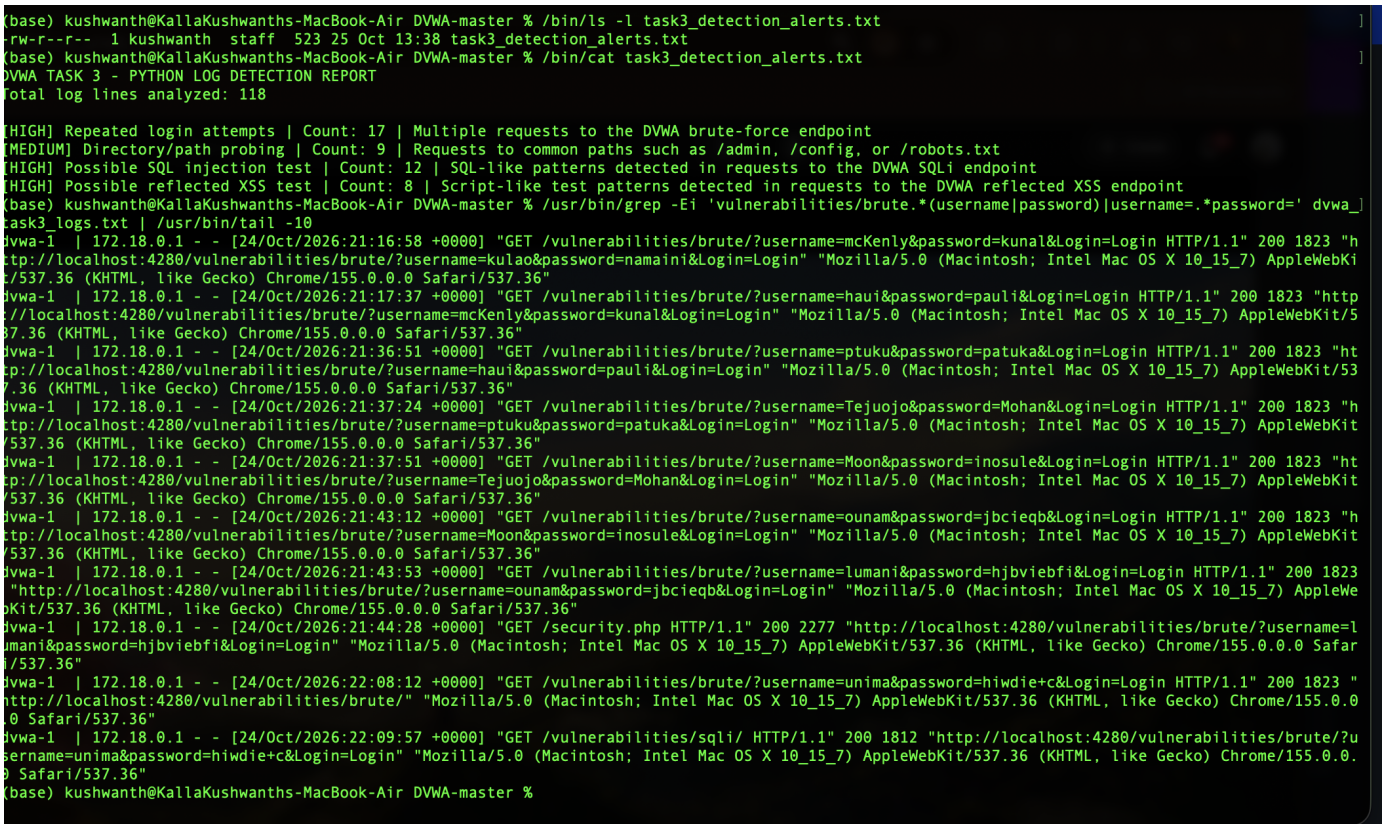



Figure E15. Saved Python detection alert summary output.

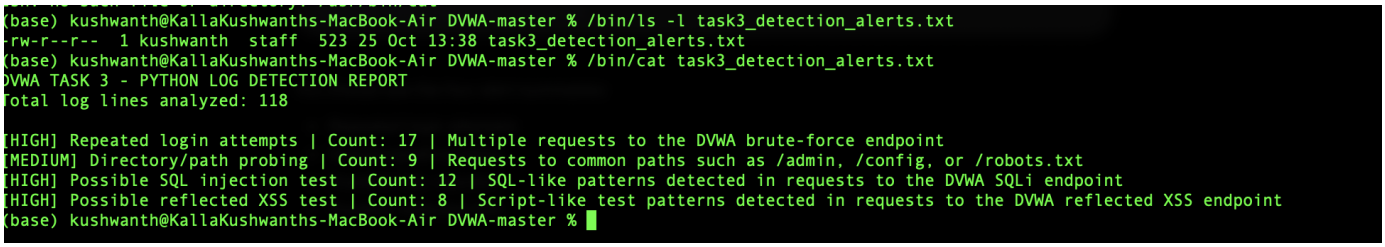



Figure E16. Terminal evidence from the detection workflow. The Python results are not Wazuh alerts.